



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Norme di Progetto

Stato	In progress
Versione	0.9.0
Autori	Davide Lorenzon Aldo Bettega Guerra Filippo Ana Maria Draghici
Verificatori	Ana Maria Draghici Davide Lorenzon Aldo Bettega
Uso	Interno
Destinatari	Tutto il gruppo

Vers.	Data	Autore	Verificatore	Descrizione
0.9.0	2026-01-10	Ana Maria Draghici	-	Rivisto processi primari Sezione 2.1 , sottosezione Sezione 2.2 e da definire processo di sviluppo
0.8.4	2025-12-15	Davide Lorenzon	-	Stesura della sezione Sezione 4.3 , Processo di miglioramento e Sezione 4.4 , processo di formazione
0.8.3	2025-12-14	Davide Lorenzon	-	Stesura della sezione Sezione 4.2 , Gestione dell'infrastruttura
0.8.2	2025-12-13	Davide Lorenzon	-	Rivista introduzione, approfondita Sezione 4.1 , gestione del processo
0.8.1	2025-12-10	Davide Lorenzon	Ana Maria Draghici	Aggiunta: Sezione 4.1.4 Rendicontazione delle ore
0.8.0	2025-12-09	Filippo Guerra	Davide Lorenzon	Aggiunta sezione 2.2: Processo di fornitura
0.7.0	2025-12-04	Aldo Bettega	Davide Lorenzon	Aggiunta sezione 4.1.2 e sezione 9. aggiornata 5.3 definition of done
0.6.1	2025-12-02	Davide Lorenzon	Davide Lorenzon	Apportate modifiche di ordine nella sottosezione documentazione
0.6.0	2025-12-02	Ana Maria Draghici	Aldo Bettega	Aggiunta sezione 4.8 «struttura specifica dei documenti» e relative sottose-

Vers.	Data	Autore	Verificatore	Descrizione
				zioni, aggiunto sezione 5.4.3 «Versionamento»
0.5.0	2025-11-30	Ana Maria Draghici	Aldo Bettega	Aggiunta sezione 5.3 e 5.4, rispettivamente «Definition of Done» e «Issue tracking System»
0.4.0	2025-11-29	Guerra Filippo	Ana Maria Draghici	Aggiunta sezione 5.2 «ruolo-documento»
0.3.0	2025-11-11	Ana Maria Draghici	Davide Lorenzon	Aggiunta sezione 4.8.1 struttura Analisi Requisiti
0.2.0	2025-11-11	Davide Lorenzon	Ana Maria Draghici	Aggiunta struttura dei documenti (come stabilito da verbale 2025-11-10)
0.1.0	2025-11-	Davide Lorenzon	Ana Maria Draghici	Stesura iniziale

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	1
1.4.1) Riferimenti normativi	1
1.4.2) Riferimenti informativi	2
2) Processi Primari	3
2.1) Introduzione ai processi primari	3
2.2) Processo di fornitura	3
2.2.1) Introduzione	3
2.2.2) Scopo del processo	3
2.2.3) Attività del processo	4
2.2.4) Procedure operative	4
2.2.5) Documenti principali	5
2.2.6) Strumenti a supporto	6
2.3) Processo di sviluppo	6
2.3.1) Introduzione	6
2.3.2) Scopo del processo	6
2.3.3) Attività del processo di sviluppo	7
2.3.4) Inquadramento del processo rispetto alle Baseline	8
2.3.5) Procedure operative	8
2.3.6) Documentazione prodotta	10
2.3.7) Strumenti a supporto	10
3) Processi di Supporto	11
3.1) Documentazione	11
3.1.1) Impegno del gruppo nella produzione della documentazione	11
3.1.2) Workflow documentale	11
3.1.2.1) Stati del documento	11
3.1.2.2) Procedura di avanzamento tra stati	12
3.1.3) Strumenti di supporto	12
3.1.4) Identificazione dei documenti	13
3.1.4.1) Elenco dei documenti	13
3.1.5) Progettazione dei documenti	13
3.1.6) Pubblicazione e Distribuzione della Documentazione	14
3.1.7) Posizione del Documento	14
3.1.8) Informazioni comuni	14
3.1.9) Documentazione prevista per la fase RTB	15
3.1.10) Struttura specifica dei documenti	15
3.1.10.1) Analisi dei Requisiti	16

3.1.10.1.1) Struttura principale	16
3.1.10.2) Piano di Progetto	17
3.1.10.2.1) Struttura principale	17
3.1.10.3) Piano di Qualifica	18
3.1.10.3.1) Struttura principale	18
3.1.10.4) Verbalì	19
3.1.10.4.1) Procedure e responsabilità	19
3.1.10.4.2) Struttura Verbale	19
3.1.10.5) Diario di Bordo	20
3.1.10.6) Norme di Progetto	21
3.1.10.6.1) Struttura principale	21
3.1.10.7) Glossario	22
3.1.10.8)	22
3.1.11) Manutenzione	22
4) Processi Organizzativi	23
4.1) Gestione del Processo	23
4.1.1) Scopo	23
4.1.2) Attività previste	23
4.1.2.1) Inizializzazione	23
4.1.2.2) Pianificazione	23
4.1.2.3) Esecuzione e Controllo	23
4.1.2.4) Revisione e valutazione	24
4.1.2.5) Chiusura	24
4.1.3) Ruoli di Progetto	24
4.1.4) Rendicontazione delle ore	25
4.1.5) Assegnazione Ruolo-Documento	25
4.1.6) Definition of Done	26
4.1.7) Issue tracking System – Guida Operativa	27
4.1.7.1) Creazione di una nuova issue	27
4.1.7.2) Flusso operativo	28
4.1.8) Versionamento	29
4.1.8.1) Codice di versione	29
4.1.8.1.1) Regole di incremento	29
4.2) Gestione dell'Infrastruttura	29
4.2.1) Attività previste	29
4.2.2) Implementazione del processo	30
4.2.3) Creazione	30
4.2.4) Manutenzione	32
4.2.4.1) Issue tracking System - Guida Operativa	32
4.2.4.2) Creazione e struttura di un issue	32
4.2.4.2.1) Flusso operativo	34

4.2.4.2.2)	Project board	34
4.2.4.2.3)	Milestone	34
4.2.4.3)	GitHub Actions	34
4.2.4.4)	Script e automazioni	34
4.2.4.5)	Discord	34
4.2.4.6)	Strumenti Google	35
4.3)	Processo di miglioramento	35
4.3.1)	inizializzazione del processo	35
4.3.2)	Valutazione del processo	35
4.3.3)	Miglioramento del processo	35
4.3.4)	Ciclo PDCA	35
4.4)	Processo di formazione	36
4.4.1)	Avvio del processo	36
4.4.2)	Sviluppo	36
4.4.3)	Esecuzione	36
4.4.4)	Apprendimento libero	36
5)	Metriche e standard per la Qualità	37
6)	Metriche di Qualità del Processo	38
7)	Metriche di Qualità del Prodotto	39
8)	Best Practices	40
8.1)	Formato nome dei verbali	40
8.2)	Scrittura dei commit	40
8.3)	Issue Template	40

1) Introduzione

1.1) Scopo del documento

In questo documento il gruppo definisce e mantiene traccia del proprio way of working, ovvero l'insieme di pratiche e convenzioni adottate per organizzare e svolgere il lavoro di progetto.

La stesura del documento avviene in modo incrementale, evolvendosi parallelamente all'avanzamento delle attività. Nel corso del progetto potrà essere soggetto ad aggiunte, modifiche o rimozioni, derivate dal processo di apprendimento e sperimentazione del team.

Tali aggiornamenti nascono dall'analisi e dall'evoluzione delle best practices comuni nell'ingegneria del software, che il gruppo studia, applica e adatta in funzione delle esigenze del team e del contesto progettuale.

1.2) Scopo del prodotto

Il prodotto sviluppato è un'applicazione software progettata per verificare in modo automatico la conformità alla norma EN 18031, lo standard tecnico europeo che definisce i requisiti di sicurezza informatica per i dispositivi radio wireless (es. Wi-Fi, LTE, Bluetooth, IoT wireless).

L'obiettivo dell'applicazione è supportare e guidare l'utente nella valutazione dei requisiti di cybersecurity, eseguendo in autonomia i percorsi decisionali tramite decision tree (alberi di decisione). Questo approccio permette di accelerare, standardizzare e rendere più affidabile il processo di verifica della conformità, con la generazione automatica della documentazione tecnica richiesta a supporto dell'assessment.

1.3) Glossario

Per garantire precisione terminologica senza compromettere la leggibilità, in questo documento viene adottato un approccio ibrido alla gestione dei riferimenti al Glossario. I termini tecnici possono essere presentati secondo 2 modalità:

- **Footnote al primo utilizzo:** applicata ai concetti critici o potenzialmente ambigui, permette un accesso immediato alla definizione senza interrompere il flusso logico del testo.
- **Marcatura tramite pedice "G" (termine _G):** utilizzata per termini ricorrenti o già contestualizzati, indica semplicemente la presenza del termine nel Glossario.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Software Engineering, Ian Sommerville](#)
- [Regolamento progetto](#)

- [Capitolato d'appalto Automated EN18031 Compliance Verification di BlueWind](#)
- [Standard ISO 12207:2017](#)

1.4.2) Riferimenti informativi

2) Processi Primari

2.1) Introduzione ai processi primari

Per sviluppare un sistema software di qualità, la semplice scrittura di codice e l'esecuzione dei test non sono sufficienti: il prodotto deve essere **affidabile, utilizzabile da un ampio numero di utenti e facilmente manutenibile nel tempo**.

Per raggiungere questi obiettivi, è necessario adottare un modello di sviluppo che definisca **processi chiari e strutturati**, capaci di guidare in modo sistematico le attività del progetto.

Secondo lo **standard ISO/IEC 12207**, i principali processi primari sono:

- **Fornitura**, che gestisce i rapporti con il committente o proponente esterno;
- **Sviluppo**, che si occupa della realizzazione del prodotto software vero e proprio.

2.2) Processo di fornitura

2.2.1) Introduzione

Il processo di fornitura descrive le attività necessarie per **instaurare, gestire e mantenere il rapporto tra il gruppo e la proponente esterna (BlueWind S.r.l.)** durante lo svolgimento del progetto.

Seguendo le linee guida dettate dai principi espressi dallo **standard ISO/IEC 12207:1997** — che definisce il processo di supply come l'insieme delle attività volte alla definizione degli accordi contrattuali, alla pianificazione, al monitoraggio e alla consegna dei prodotti software — il gruppo si impegna a stabilire modalità **organizzate, verificabili e tracciabili** per comunicare, consegnare documentazione e garantire trasparenza verso la proponente.

2.2.2) Scopo del processo

Lo scopo del processo di fornitura è garantire che la relazione tra gruppo e proponente sia **strutturata, efficiente e basata su comunicazioni chiare e documentate**.

Gli obiettivi principali includono:

- Definire i **canali ufficiali di comunicazione** tra gruppo e proponente;
- Stabilire **frequenza, modalità e tempi dei confronti**;
- Assicurare che le richieste della proponente siano **comprese, tracciate e integrate** nel flusso di lavoro;
- Produrre **documentazione coerente** per revisioni e fasi di progetto;
- Garantire **trasparenza, tracciabilità e rispetto delle scadenze** previste da capitolato e milestone.

2.2.3) Attività del processo

Il processo di fornitura comprende le seguenti attività principali:

1. **Inizializzazione**

Analisi delle richieste della proponente, identificazione dei requisiti contrattuali e valutazione dei vincoli organizzativi.

2. **Preparazione delle risposte**

Realizzazione di eventuali contro-proposte basate sull'analisi dei requisiti.

3. **Contrattazione**

Confronto con la proponente per formalizzare requisiti, scadenze e modalità di lavoro.

4. **Pianificazione**

Definizione dell'organizzazione, del ciclo di vita del software, assegnazione delle risorse e gestione dei rischi.

5. **Esecuzione e controllo**

Implementazione delle attività pianificate, monitoraggio della qualità e del progresso del lavoro.

6. **Revisione e valutazione**

Raccolta di feedback dalla proponente per eventuali correzioni o aggiornamenti del lavoro.

7. **Consegna e completamento**

Rilascio dei prodotti software e della documentazione, garantendo supporto post-consegna.

2.2.4) Procedure operative

Per garantire continuità e tracciabilità nelle comunicazioni, il gruppo utilizza modalità **sincrone e asincrone**:

Comunicazione sincrona (meeting):

- **Riunioni con la proponente (BlueWind S.r.l.):**
- Programmate periodicamente secondo le esigenze del progetto.
- Documentate tramite verbali esterni.
- Decisioni, richieste e dubbi vengono registrati come issue su GitHub.
- **Riunioni interne al gruppo:**
- Programmate per coordinare attività e aggiornare lo stato del progetto.
- Documentate tramite verbali interni.
- Decisioni operative e compiti interni tracciati come issue su GitHub.

Comunicazione asincrona:

- **Con la proponente:** email ufficiale, messaggistica (Telegram) per chiarimenti rapidi.
- **Internamente al gruppo:** chat interne (Discord/WhatsApp) per coordinamento e aggiornamenti.

Tracciamento

- Tutte le comunicazioni rilevanti vengono registrate;
- Verbali esterni e interni (questi ultimi per riunioni del gruppo senza la presenza della proponente);
- Issue GitHub per decisioni operative e richieste tracciate.

Disponibilità

Il gruppo si impegna a fornire aggiornamenti regolari e materiali richiesti entro le scadenze di sprint e milestone. Il processo di fornitura produce documentazione fondamentale per la tracciabilità e la trasparenza del progetto.

2.2.5) Documenti principali

- **Verbali esterni:** registrazione dei meeting con BlueWind;
- **Verbali interni:** documentazione di riunioni interne;
- **Norme di Progetto (NdP):** descrizione dei processi adottati e dell'organizzazione interna;
- **Piano di Progetto (PdP):** pianificazione delle attività, disponibilità delle risorse, avanzamento del lavoro;
- **Analisi dei Requisiti (AdR):** raccolta e formalizzazione dei requisiti della proponente;
- **Piano di Qualifica (PdQ):** criteri di verifica e validazione, test effettuati e risultati;
- **Glossario:** definizione dei termini tecnici e concetti chiave utilizzati nel progetto;
- **Dichiarazione degli Impegni:** stima dei costi del progetto, ore per ruolo e responsabilità dei componenti del gruppo;
- **Lettera di Candidatura:** presentazione ufficiale della candidatura del gruppo al capitolato proposto;
- **Valutazione dei Capitolati:** analisi dei capitolati disponibili, punti di forza, criticità e motivazioni della scelta effettuata dal gruppo.

2.2.6) Strumenti a supporto

Per svolgere le attività del processo di fornitura, il gruppo utilizza strumenti sia interni sia esterni:

Strumenti interni

- **GitHub**: gestione backlog, ticketing;
- **Google Calendar**: gestione appuntamenti e scadenze;
- **Discord / Whatsapp** : coordinamento interno e riunioni del gruppo.

Strumenti verso la proponente

- **Google Mail**: comunicazioni ufficiali scritte;
- **Zoom**: riunioni sincrone remote.
- **Telegram** : indicata dalla proponente per chiarimenti rapidi.

2.3) Processo di sviluppo

2.3.1) Introduzione

Il processo di sviluppo descrive l'insieme delle attività necessarie alla realizzazione del prodotto software, a partire dall'analisi dei requisiti fino alla consegna e accettazione del sistema finale.

Secondo lo standard **ISO/IEC 12207**, il processo di sviluppo comprende tutte le attività tecniche volte a trasformare i requisiti concordati con il committente in un prodotto software funzionante, verificato e conforme agli obiettivi di qualità stabiliti. Tali attività includono l'analisi dei requisiti, la progettazione dell'architettura, la codifica, l'integrazione, il testing e la validazione del sistema.

Nel contesto del progetto, il processo di sviluppo è adottato per garantire che ogni fase di realizzazione del software sia svolta in modo sistematico, tracciabile e coerente con le specifiche definite, assicurando la qualità, l'affidabilità e la manutenibilità del prodotto nel tempo.

2.3.2) Scopo del processo

Lo scopo del processo di sviluppo è garantire la corretta realizzazione del prodotto software in conformità ai requisiti funzionali, di qualità e di vincolo definiti nel capitolato e nell'Analisi dei Requisiti.

In particolare, il processo di sviluppo ha l'obiettivo di:

- **Trasformare i requisiti approvati in una soluzione software** progettata e implementata correttamente;
- **Assicurare la tracciabilità** tra requisiti, componenti progettuali, codice e test;

- Garantire che il software **soddisfi gli standard di qualità** stabiliti;
- Individuare e correggere tempestivamente eventuali difetti attraverso **attività di verifica e validazione**;
- Produrre un sistema software **affidabile, manutenibile e conforme alle aspettative** della proponente.

Il processo di sviluppo costituisce quindi il fulcro tecnico del progetto e guida in modo strutturato tutte le attività necessarie alla costruzione del prodotto finale.

2.3.3) Attività del processo di sviluppo

Il processo di sviluppo è articolato in un insieme di attività tra loro correlate, definite in conformità allo **standard ISO/IEC 12207**, che guidano la realizzazione del prodotto software lungo l'intero ciclo di vita.

Le principali attività previste sono le seguenti:

1. **Analisi dei requisiti**

Attività volta all'identificazione, analisi e formalizzazione dei requisiti funzionali, di qualità e di vincolo del sistema. I requisiti vengono raccolti a partire dal capitolato e dal confronto con la proponente, documentati nell'Analisi dei Requisiti e resi tracciabili per le successive fasi di progettazione, implementazione e verifica.

2. **Progettazione dell'architettura del sistema**

Definizione della struttura generale del sistema, individuando le componenti principali, le loro responsabilità e le interazioni tra di esse, al fine di soddisfare i requisiti individuati.

3. **Progettazione dell'architettura software**

Scomposizione del sistema in componenti software e moduli, con definizione delle interfacce e delle dipendenze, mantenendo la coerenza con l'architettura di sistema e garantendo la tracciabilità dei requisiti.

4. **Progettazione di dettaglio**

Definizione dettagliata delle singole componenti software e delle unità che le compongono, fornendo le informazioni necessarie alla fase di codifica.

5. **Codifica**

Implementazione del software secondo quanto definito in fase di progettazione, adottando convenzioni di stile e buone pratiche di programmazione per garantire leggibilità, manutenibilità e qualità del codice.

6. **Testing delle unità e integrazione del software**

Verifica del corretto funzionamento delle singole unità software e successiva integrazione delle componenti, accompagnata da test di integrazione per individuare eventuali difetti.

7. Verifica e validazione del sistema

Esecuzione dei test di qualifica del software e del sistema per verificare la conformità ai requisiti e agli obiettivi di qualità definiti nel Piano di Qualifica.

8. Installazione e supporto all'accettazione

Consegna del prodotto software nell'ambiente concordato e supporto alla proponente nelle attività di accettazione, al fine di verificare il soddisfacimento dei requisiti contrattuali.

2.3.4) Inquadramento del processo rispetto alle Baseline

Il processo di sviluppo è strettamente collegato alle baseline previste dal progetto:

Requirements and Technology Baseline (RTB)

Comprende principalmente le attività di:

- Analisi dei Requisiti;
- Prime attività di codifica e prototipazione, ove previste.

Product Baseline (PB)

Comprende principalmente le attività di:

- Progettazione dell'architettura di sistema;
- Progettazione dell'architettura software;
- Codifica completa del prodotto.

2.3.5) Procedure operative

Le attività del processo di sviluppo vengono svolte seguendo procedure operative standardizzate, al fine di garantire coerenza, tracciabilità e qualità del prodotto software.

Ogni procedura è supportata e documentata tramite i principali artefatti di progetto: Analisi dei Requisiti (AdR), Piano di Progetto (PdP), Piano di Qualifica (PdQ) e Norme di Progetto (NdP).

Analisi dei requisiti

- Analizzare il capitolato e la documentazione fornita dalla proponente;
- Raccogliere eventuali chiarimenti tramite comunicazioni ufficiali;
- Identificare gli attori del sistema e le principali interazioni con il software;
- Individuare e descrivere i casi d'uso, specificando per ciascuno
 1. attori coinvolti;
 2. scenario principale;
 3. scenari alternativi ed eccezioni;
- Assegnare a ogni caso d'uso un identificativo univoco secondo una nomenclatura coerente;
- Derivare dai casi d'uso e dal capitolato i requisiti del sistema;
- Classificare i requisiti in:

1. requisiti funzionali;
 2. requisiti di qualità;
 3. requisiti di vincolo;
- Assegnare a ciascun requisito:
 1. un identificativo univoco;
 2. una priorità (obbligatorio, desiderabile, opzionale);
 - Formalizzare casi d'uso e requisiti nel documento di Analisi dei Requisiti (AdR);
 - Associare ogni requisito a uno o più casi d'uso;
 - Garantire la tracciabilità dei requisiti verso le successive attività di progettazione, codifica e verifica, come riportato in AdR e PdQ.

Progettazione

- Definire l'architettura generale del sistema sulla base dei requisiti approvati;
- Individuare le componenti software e le loro responsabilità;
- Definire le interfacce tra le componenti;
- Aggiornare la documentazione di progetto in base alle decisioni architetturali;
- Verificare la coerenza tra requisiti definiti in AdR e le scelte progettuali;
- Allineare le attività progettuali alla pianificazione definita nel Piano di Progetto (PdP).

Codifica

- Implementare le funzionalità secondo la progettazione approvata;
- Applicare le convenzioni di stile e le buone pratiche definite nelle Norme di Progetto;
- Utilizzare il sistema di versionamento per la gestione del codice;
- Suddividere il lavoro in attività pianificate nel PdP e tracciate tramite issue;
- Eseguire controlli statici, formattazione automatica e verifiche preliminari del codice;
- Garantire la tracciabilità tra codice implementato e requisiti definiti in AdR.

Verifica e test

- Definire i casi di test in conformità al Piano di Qualifica (PdQ);
- Eseguire test di unità sulle singole componenti software;
- Eseguire test di integrazione tra le componenti;
- Registrare l'esito dei test, le non conformità e le eventuali correzioni nel PdQ;
- Verificare la copertura dei requisiti definiti in AdR attraverso i test eseguiti.

Integrazione e rilascio

- Integrare progressivamente le componenti software secondo la pianificazione del PdP;
- Verificare il corretto funzionamento del sistema nel suo insieme;
- Eseguire i test di qualifica di sistema previsti dal PdQ;
- Preparare il materiale di rilascio e la documentazione associata;

- Effettuare la consegna secondo le modalità e le scadenze concordate con la proponente.

Gestione delle modifiche

- Registrare le richieste di modifica come issue;
- Valutare l'impatto delle modifiche sui requisiti (AdR) e sulla pianificazione (PdP);
- Aggiornare la documentazione di progetto e il codice;
- Eseguire nuovamente le verifiche previste dal PdQ;
- Garantire il mantenimento della tracciabilità tra requisiti, codice e test.

2.3.6) Documentazione prodotta

Durante il processo di sviluppo, le attività sono supportate dalla documentazione di progetto già definita nel processo di fornitura (AdR, PdP, PdQ, NdP), con l'aggiunta di riferimenti specifici alle fasi di implementazione e verifica software.

2.3.7) Strumenti a supporto

Per lo sviluppo del software, il gruppo utilizza strumenti mirati a garantire qualità, tracciabilità e collaborazione:

- **Linguaggio e ambiente di sviluppo:** Python 3.x come linguaggio principale per le componenti software.
- **Versionamento del codice:** Git/GitHub per gestione dei repository, branch, commit, issue e pull request.
- **Formattazione e controllo del codice:**
- **Gestione attività e tracciamento:** GitHub Issues per assegnazione, monitoraggio e gestione delle modifiche.
- **Comunicazione e collaborazione interna:** Discord o WhatsApp per coordinamento rapido, aggiornamenti sullo stato di avanzamento e chiarimenti tra membri del gruppo.
- **Comunicazione verso la proponente:** email ufficiale, Zoom o Teams per riunioni sincrone e Telegram per chiarimenti rapidi.

3) Processi di Supporto

3.1) Documentazione

Il processo di documentazione ha lo scopo di registrare e rendere disponibili le informazioni prodotte dai processi primari del progetto.

Consente di tracciare con maggiore efficacia le decisioni adottate, ridurre il rischio di ambiguità, evitare fraintendimenti e facilitare l'organizzazione del **lavoro asincrono e collaborativo**.

Le principali attività che compongono questo processo sono:

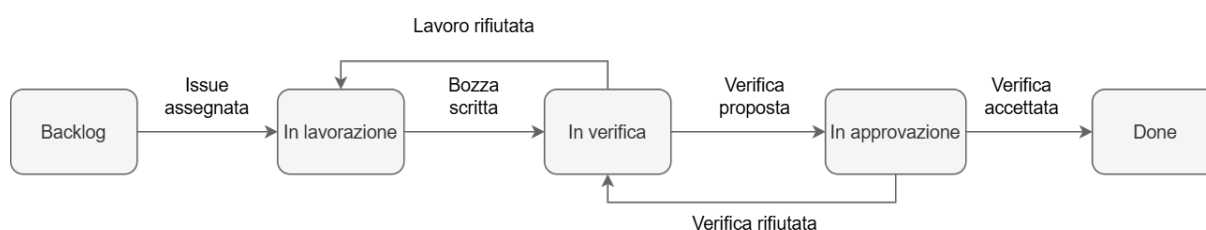
1. **Identificazione dei documenti** → individuazione dei documenti necessari e ne riferisce
2. **Progettazione dei documenti** → applicazione delle norme e del workflow stabilito;
3. **Pubblicazione e Distribuzione della Documentazione** → generazione del documento in formato destinato alla distribuzione;
4. **Manutenzione** → aggiornamento continuo del contenuto e gestione delle modifiche;
5. **Archiviazione e tracciabilità** → versionamento, conservazione e accessibilità nel tempo.

3.1.1) Impegno del gruppo nella produzione della documentazione

Il gruppo si impegna a fornire alla proponente e ai docenti tutta la documentazione necessaria a supportare le attività di analisi, progettazione, sviluppo e verifica del progetto. Tale documentazione ha lo scopo di garantire trasparenza, tracciabilità e qualità del lavoro svolto, oltre a costituire un riferimento chiaro per tutti gli stakeholder coinvolti.

3.1.2) Workflow documentale

All'interno dell'ambito documentale è stato optato il seguente modello per descrivere e modellare le attività necessarie a produrre un documento:



3.1.2.1) Stati del documento

- **Backlog**: magazzino delle attività da svolgere, ogni documento inizia in questo stato.

- **In lavorazione:** il documento è stato preso in carico da un autore.
- **In verifica:** Il lavoro dell'autore è terminato. Il documento deve ora essere revisionato oppure corretto, nel caso in cui non sia stato approvato durante la fase di validazione.
- **In approvazione,** il lavoro del revisore è finito. Il documento va valutato per l'approvazione oppure respinto, fornendo le opportune motivazioni accompagnate da un elenco delle correzioni da apportare.
- **Done,** il documento è stato approvato.

3.1.2.2) Procedura di avanzamento tra stati

- Da **Backlog** a **In lavorazione:** un autore si assegna una issue e inizia a scrivere la bozza del documento.
- Da **In lavorazione** a **In verifica:** l'autore consegna la bozza, trasferendo la issue in revisione e assegnandola al revisore (deciso a priori) che verrà notificato automaticamente.
- Da **In verifica** a **In validazione:** il revisore ha apportato modifiche alla bozza e propone la revisione al validatore. Il revisore deve spostare la issue in validazione e assegnarla al validatore.
- Da **In validazione** a **In verifica:** il validatore rifiuta la revisione proposta allegando una lista di modifiche motivate che il revisore dovrà apportare al documento. Il validatore dovrà riassegnare la issue al revisore.
- Da **In validazione** a **Done:** il validatore accetta la revisione proposta e chiude la issue con

```
git commit -m "commento. Close #numero_issue"
```

3.1.3) Strumenti di supporto

Typst: Linguaggio di markup moderno per la composizione e la tipografia di documenti, pensato come alternativa più semplice e veloce a LaTeX.

- Sintassi intuitiva;
- Supporto ad automazioni tramite template, funzioni e regole di stile riutilizzabili;
- Preview istantanea del documento.

Github: Strumento scelto dal gruppo per la condivisione del lavoro e la gestione delle attività tramite **issue tracking**.

- Utilizzo di GitHub Actions per la compilazione automatica dei documenti.
- Documentazione disponibile nel repository [Github](#).
- [Sito web](#) predisposto tramite GitHub Pages per facilitare la consultazione della documentazione.

TurboScribe AI: Per velocizzare il processo, il gruppo utilizza il tool AI [TurboScribe](#) che consente di trascrivere facilmente da registrazioni audio, utile per funzionalità come il riascolto di momenti specifici della riunione tramite selezione testuale.

3.1.4) Identificazione dei documenti

Si tratta di una fase di pianificazione in cui ogni documento viene definito secondo le seguenti caratteristiche principali:

- Titolo;
- Scopo;
- Destinatari;
- Procedure e responsabilità nella redazione e gestione del documento;
- Pianificazione per le versioni e i loro contenuti.

3.1.4.1) Elenco dei documenti

Documento 1	Analisi dei Requisiti	16
Documento 2	Piano di Progetto	17
Documento 3	Piano di Qualifica	18
Documento 4	Verbali	19
Documento 5	Diario di Bordo	20
Documento 6	Norme di Progetto	21
Documento 7	Glossario	22
Documento 8		22

3.1.5) Progettazione dei documenti

Ogni documento identificato all'interno dello sviluppo software deve rispettare alcuni standard di documentazione uguali per tutti:

- Essere in formato A4;
- I contenuti inseriti devono essere coerenti con lo scopo del documento stesso;
- Tutti i documenti devono includere un indice dei contenuti e delle relative sotto-sezioni visibile all'inizio (ad eccezione del diario di bordo, che ne è esentato);
- Ogni pagina deve contenere nell'header e nel footer:
 1. La sezione corrente del documento (in alto a sinistra);
 2. Il nome del gruppo (in alto a destra);
 3. Il titolo del documento (in basso a sinistra);
 4. Il numero della pagina : espresso in numeri romani per la prefazione e in numeri arabi per le pagine del corpo del documento.

Per la redazione dei documenti è corretto e necessario fare riferimento a fonti autorevoli e aggiornate, quali standard ISO, università e organizzazioni ufficialmente riconosciute, materiale didattico e link di approfondimento forniti durante il corso. Eventuali informazioni provenienti da altre fonti, se ritenute utili, devono essere obbligatoriamente verificate per accertarne la correttezza e l'affidabilità.

3.1.6) Pubblicazione e Distribuzione della Documentazione

I documenti vengono ricompilati automaticamente in PDF tramite GitHub Action e sono consultabili da tutti i membri del team. Sono disponibili nella repository del gruppo dedicata alla [Documentazione](#). La posizione ufficiale dei file è indicata nel README.md del repository.

Per facilitarne la consultazione è inoltre possibile accedere al [sito web ufficiale](#) del gruppo, creato appositamente per visualizzare e navigare i documenti in modo più immediato.

La versione del documento e la tracciabilità delle modifiche sono gestite tramite il Registro delle Modifiche, integrato direttamente all'interno di ciascun documento.

3.1.7) Posizione del Documento

I documenti sono salvati sull'apposito repository.

Il path relativo è ricavabile nel seguente modo:

(Fanno eccezione i diari di bordo, in quanto fanno parte delle regole di progetto, ma non fanno supporto ad alcun processo primario, perciò sono salvati nella cartella ./<Type>src/DiariDiBordo)

./<Type>/<Milestone>/<Destinatari>/<Cartella del Documento>

Type:

- **src** per i file in formato typst.
- **output** per i pdf.

Milestone:

- **RTB** per i documenti allo stato della RTB.
- **PB** per i documenti allo stato della PB.

Destinatari:

- DocumentazioneInterna per i documenti ad uso interno.
- DocumentazioneEsterna per i documenti ad uso esterno.

Cartella del Documento:

- VerbalInterni
- VerbalEsterni
- Per documenti complessi coincide con il nome del documento¹

3.1.8) Informazioni comuni

Ogni documento presenta una sezione iniziale standardizzata per tutti i membri del team. Questa sezione viene generata utilizzando un apposito template centrale e unico, al fine di garantire coerenza e facilitare la compilazione.

La sezione iniziale è composta dai seguenti elementi:

¹Per motivi di manutenibilità e facilità di aggiornamento i contenuti del file sono stati divisi in più file quando una loro sezione diventa eccessivamente corposa.

1. **Pagina di copertina** contenente il titolo del documento, il nome e il logo del gruppo e le relative informazioni di contatto.
2. **Tabella dello stato** che riassume lo stato del documento e informazioni generali quali versione, autori, verificatori, uso e destinatari.
3. **Registro delle modifiche** costituito da una tabella contenente le informazioni sul versionamento e sulla tracciabilità.
4. **Indice dei contenuti** aggiornato automaticamente tramite sintassi Typst.
5. **Indice delle immagini e delle tabelle** presente solo nei documenti che ne contengono.

3.1.9) Documentazione prevista per la fase RTB

La documentazione fornita in corrispondenza della fase RTB (Requirements and Technology Baseline) comprende sia materiali tecnici sia documenti operativi e organizzativi, al fine di garantire una valutazione completa e trasparente dello stato del progetto.

In particolare, vengono prodotti:

Documenti tecnici :

- Analisi dei Requisiti (AdR);
- Piano di Progetto (PdP);
- Piano di Qualifica (PdQ);
- Preventivo dei Costi e delle risorse impiegate;

Documenti operativi e organizzativi

- Norme di Progetto (NdP);
- Verbali interni;
- Verbali esterni;
- Glossario;
- Diario di bordo.

3.1.10) Struttura specifica dei documenti

Di seguito viene riportata la struttura standard dei documenti principali, le rispettive sezioni, il loro scopo, i destinatari, e le metodologie adottate per la scrittura e la revisione, al fine di mantenere coerenza e uniformità all'interno del gruppo.

3.1.10.1) Analisi dei Requisiti

L'Analisi dei Requisiti ha il compito di descrivere in modo completo, chiaro e verificabile tutte le funzionalità che il sistema deve offrire, includendo sia requisiti funzionali sia non funzionali. Il documento fornisce inoltre i principali casi d'uso, con attori e scenari associati, e garantisce la tracciabilità tra requisiti, casi d'uso ed eventuali estensioni future. Rappresenta un riferimento stabile per sviluppatori, tester e manutentori durante tutte le fasi del progetto.

Destinatari : stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.1.10.1.1) Struttura principale

Il documento comprende:

- Definizione formale dei requisiti funzionali e non funzionali;
- Modellazione dei casi d'uso con attori e flussi narrativi;
- Matrice di tracciabilità requisiti-casi d'uso;
- Eventuali vincoli tecnici, operativi o di contesto.

Documento 1: Analisi dei Requisiti

3.1.10.2) Piano di Progetto

Il Piano di Progetto definisce la pianificazione complessiva delle attività, descrivendo l'approccio plan-driven adottato dal gruppo. Il documento fornisce una visione strutturata dell'organizzazione del lavoro, includendo la definizione degli obiettivi, la gestione delle risorse, l'assegnazione dei ruoli, la pianificazione temporale e l'analisi dei rischi. La sua funzione principale è quella di garantire un monitoraggio costante dell'avanzamento del progetto attraverso revisioni periodiche e rendicontazioni relative ai vari sprint. Tale monitoraggio consente al gruppo di valutare l'efficienza del workflow, di individuare tempestivamente criticità e di adattare la pianificazione quando necessario.

Destinatari : stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.1.10.2.1) Struttura principale

La struttura del documento comprende:

- Ambito e obiettivi del progetto;
- Preventivo iniziale e disponibilità delle risorse;
- Analisi dei rischi e piano di mitigazione;
- Distribuzione dei ruoli e pianificazione temporale;
- Sezione dedicata alle revisioni, con per ogni sprint:
 - **Retrospettiva del gruppo** → riflessioni su apprendimento, workflow ed efficacia della collaborazione
 - **Attività pianificate** → obiettivi e task previsti per il periodo di sprint
 - **Rischi e difficoltà emersi** → analisi degli impedimenti riscontrati e strategie di mitigazione
 - **Preventivo ore per ruolo** → stima dell'effort pianificato, suddiviso per responsabilità
 - **Consuntivo ore effettive** → ore realmente impiegate dal gruppo nel periodo

Documento 2: Piano di Progetto

3.1.10.3) Piano di Qualifica

Il piano di qualifica ha l'obiettivo di garantire che il prodotto sviluppato rispetti elevati standard di qualità. Definisce processi, metriche, strategie di testing e criteri di valutazione necessari a verificare la qualità del software e del processo di sviluppo. Fornisce inoltre strumenti operativi per la misurazione e la validazione dei risultati.

Destinatari: stakeholder interni ed esterni al progetto (BlueWind S.r.l., docenti e gruppo interno)

3.1.10.3.1) Struttura principale

Il documento è articolato nelle seguenti componenti:

- Metriche di qualità del prodotto e del processo;
- Strategie, livelli e tecniche di testing;
- Piano delle verifiche e validazioni;
- Cruscotto qualità con indicatori e soglie di accettazione;

Documento 3: Piano di Qualifica

3.1.10.4) Verbali

I verbali sono suddivisi in due categorie principali :

- Verbali interni -> documentano riflessioni e confronti avvenuti esclusivamente tra i membri del gruppo.
- Verbali esterni -> vengono redatti in corrispondenza di riunioni o confronti con l'azienda di riferimento (Bluewind).

Ogni verbale si conclude con una **riflessione finale del gruppo**, dalla quale emergono decisioni operative che vengono successivamente formalizzate tramite la creazione di **issue GitHub**, che il gruppo si impegna a completare.

3.1.10.4.1) Procedure e responsabilità

Il verbale deve essere un riassunto chiaro e oggettivo della riunione. Per velocizzare il processo, il gruppo utilizza il tool AI [TurboScribe](#).

3.1.10.4.2) Struttura Verbale

Ogni verbale deve avere la seguente suddivisione numerata:

1. **Informazioni comuni della sezione 4.1.2.1** (standard condivisi di documento)
2. **Informazioni generali**
 - Data e luogo della riunione
 - Orario di inizio/fine
 - Partecipanti
 - Tipo(interno/esterno)
 - Motivo (principalmente per verbali esterni)
 - Scriba (responsabile del verbale in quel momento)
3. **Ordine del giorno**
 - Scaletta dei temi da discutere, raccolti e organizzati del responsabile sulla base dei contributi dei membri del gruppo o dei referenti aziendali.
4. **Riassunto della riunione**
 - Sintesi breve e oggettiva dei punti discussi.
5. **Decisioni**
 - Azioni o obiettivi (anche ad alto livello) che il gruppo deve intraprendere per dare seguito alla riunione.
6. **TODO**
 - Attività specifiche derivate dalle decisioni. Una singola decisione può essere suddivisa in più TODO, che complessivamente consentono di raggiungere l'obiettivo stabilito.

3.1.10.5) Diario di Bordo

Il Diario di bordo è un'attività prevista dal Prof. Tullio Vardanega all'interno del progetto di Ingegneria del Software. Rappresenta un momento di condivisione in cui ciascun gruppo espone il proprio stato di avanzamento, con particolare attenzione a dubbi o problematiche emerse durante lo svolgimento delle attività.

Composto principalmente da:

Titolo: Diario di bordo seguito dal numero progressivo associato.

Scopo: Fornire ai gruppi un feedback sulle attività svolte e consentire di portare all'attenzione comune eventuali dubbi relativi al processo di lavoro.

Destinatari: Prof. Tullio Vardanega e gli altri gruppi coinvolti nel progetto.

Documento 5: Diario di Bordo

3.1.10.6) Norme di Progetto

Le Norme di Progetto definiscono l'insieme di regole, convenzioni e standard adottati dal gruppo al fine di garantire coerenza, qualità e uniformità nella produzione della documentazione, del codice e dei deliverable. Il documento stabilisce inoltre procedure condivise per redazione, revisione, versionamento, gestione dei file e comunicazione interne, riducendo il rischio di errori, fraintendimenti o incoerenze operative tra i membri del team.

Destinatari : Tutti i membri del gruppo di progetto

3.1.10.6.1) Struttura principale

Il documento delle Norme di Progetto è organizzato in sezioni facilmente consultabili riassunte in:

- **Convezioni sui documenti:** regole relative al formato, all'impostazione grafica, alle intestazioni e ai footer, alla numerazione delle pagine, nonché agli stili e ai template Typst adottati dal gruppo.
- **Processo di redazione e revisione:** descrizione dei ruoli coinvolti, della responsabilità, dell'iter di approvazione dei documenti, della gestione delle modifiche e del versionamento.
- **Standard di codifica e naming:** linee guida per la definizione dei nomi di file, classi, funzioni e repository, qualora pertinenti allo sviluppo software.
- **Procedure operative comuni:** indicazioni sulle modalità di collaborazione all'interno del team, sull'uso degli strumenti (GitHub, Issue Tracking, AI tools), sulle politiche di backup e sulle modalità di archiviazione.
- **Tracciabilità e registri:** norme per mantenere aggiornato e verificabile il Registro delle Modifiche.
- **Definizione della qualità minima:** criteri oggettivi necessari per l'accettazione dei documenti e dei deliverable, utilizzati come riferimento durante verifiche e revisioni.

Documento 6: Norme di Progetto

3.1.10.7) Glossario

Il glossario ha l'obiettivo di garantire chiarezza e uniformità nelle comunicazioni, sia verso l'esterno sia all'interno del gruppo di lavoro. Raccoglie i termini ritenuti non banali, le abbreviazioni e gli acronimi utilizzati nei documenti di progetto, fornendone una definizione precisa per evitare ambiguità interpretative.

Il documento è soggetto ad aggiornamento continuo, così da riflettere l'evoluzione del progetto e mantenere allineata la terminologia utilizzata dal gruppo. Oltre alla versione inclusa nei documenti ufficiali, il team ha definito anche un glossario interno accessibile tramite il website ufficiale della documentazione, che funge da riferimento centralizzato e sempre aggiornato.

Per favorire la tracciabilità e semplificare la consultazione, nei singoli documenti tutte le parole presenti nel glossario sono evidenziate. Tale evidenziazione permette agli utenti di riconoscere immediatamente i termini che dispongono di una definizione formale, facilitando il rimando al glossario in caso di dubbi o potenziali ambiguità.

Documento 7: Glossario

3.1.10.8)

Documento 8:

3.1.11) Manutenzione

4) Processi Organizzativi

4.1) Gestione del Processo

4.1.1) Scopo

La **gestione dei processi** ha l'obiettivo di individuare le attività, i compiti da svolgere e i ruoli ai quali questi saranno assegnati.

Stabilisce come un processo vada gestito e monitorato.

Nonché permettere una comunicazione interna ed esterna efficace.

4.1.2) Attività previste

Lo standard [ISO ISO/IEC/IEEE 12207:1997](#) individua le seguenti attività.

4.1.2.1) Inizializzazione

È la prima fase del processo.

Vanno stabiliti i requisiti di ogni processo che sta venendo analizzato. Una volta stabiliti i requisiti, il responsabile ne valuta la fattibilità in base alle risorse disponibili.

4.1.2.2) Pianificazione

Il responsabile deve pianificare le attività del processo.

I piani devono contenere la descrizione delle attività e dei task associati, oltre a descrivere il prodotto software.

I piani devono contenere le seguenti informazioni:

- La tabella di marcia per il completamento dei task;
- Stima dello sforzo;
- Risorse necessarie;
- Assegnazione del compito;
- Assegnazione delle responsabilità, maggiori dettagli alla **Sezione 4.1.3**
- Quantificazione dei rischi;
- Metriche di controllo della qualità;
- Costi associati al processo di esecuzione;
- Fornitura di ambiente e infrastrutture.

4.1.2.3) Esecuzione e Controllo

Il responsabile avvia le attività di processo, in modo congruo a quanto stabilito nella fase di pianificazione, e le monitora.

- **Internamente**

Controlla il progresso delle attività e ne tiene traccia.

In caso un membro del gruppo incontri problemi che rischiano di rallentare le attività deve riferirlo al responsabile.

- **Esternamente**

Gestisce le comunicazioni con la **proponente** e il **committente**.

4.1.2.4) Revisione e valutazione

Al completamento dell'attività il verificatore si assicura che sia conforme alle metriche di qualità stabilite.

4.1.2.5) Chiusura

Un'attività si ritiene completa dopo aver superato l'attività di verifica. Si veda la **Sezione 4.1.6** per maggiori dettagli.

La chiusura delle issue legate alle attività avviene tramite merge sul main a intervalli prefissati.

4.1.3) Ruoli di Progetto

La seguente sezione descrive le fasi della progettazione software. All'interno del team, per garantire coerenza, efficienza e qualità, ogni ruolo ha compiti specifici e interviene in momenti diversi del progetto.

Nel contesto del corso di Ingegneria del Software, tutti i membri del team devono ricoprire almeno una volta ciascun ruolo definito.

La tabella sottostante riassume in maniera chiara i compiti associati a ciascun ruolo.

Fasi di progetto

— ● **Analisi** → ● **Progettazione** → ● **Implementazione** → ● **Verifica**

Ruolo	Compiti	Presenza
Responsabile	- Coordinamento piani e scadenze - Approvazione release - Comunicazione col committente - Uso efficiente delle risorse - Redazione documenti	Tutto il progetto
Amministratore	- Garanzia efficienza strumenti - Gestione tecnologie di supporto - Verifica procedure secondo norme	Tutto il progetto
Verificatore	- Testing e validazione - Controllo qualità deliverable - Conformità ai requisiti	Tutto il progetto
Analista	- Analisi dei requisiti - Definizione bisogni del sistema - Redazione specifiche funzionali	Fase iniziale
Progettista	- Progetta architettura sistema - Design e modellazione - Traduzione requisiti in struttura tecnica	Dopo analisi

Ruolo	Compiti	Presenza
Programmatore	- Codifica software - Implementazione design - Sviluppo funzionalità	Implementazione

4.1.4) Rendicontazione delle ore

La pianificazione e il monitoraggio delle ore produttive del progetto sono gestiti tramite il documento «Piano di progetto», in cui vengono registrate sia le ore previste sia quelle effettivamente svolte per ciascun ruolo e membro del gruppo.

La ripartizione oraria è accompagnata dai relativi costi, consentendo una visione completa delle risorse economiche impiegate. Per facilitare l'analisi e garantire trasparenza durante l'eventuale rotazione dei ruoli, il gruppo utilizza inoltre un foglio di calcolo Google Sheet in cui ogni componente può rendicontare e consultare le proprie ore, sia pianificate sia effettive. Questa organizzazione strutturata favorisce una gestione chiara, trasparente e collaborativa della distribuzione dei ruoli all'interno del team.

4.1.5) Assegnazione Ruolo-Documento

La seguente sezione chiarisce i documenti associati a ciascun ruolo.

L'assegnazione viene rappresentata tramite una **legenda** e una **tabella riassuntiva**.

Legenda :

Azioni:

- R = Redazione.
- V = Verifica.
- A = Approvazione.
- C = Contribuente.

Ruoli:

- Responsabile = RESP.
- Amministratore = AMM.
- Analista = ANL.
- Progettista = PRG.
- Programmatore = PGRmm.
- Verificatore = VRF.

Documento	RESP	AMM	ANL	PRG	PRGmm	VRF
Norme di Progetto (NdP)	R/A	R	-	-	-	V
Analisi dei Requisiti (AdR)	A	-	R	-	-	V
Piano di Progetto (PdP)	R/A	-	C (supporto rischi)	-	-	V

Documento	RESP	AMM	ANL	PRG	PRGmm	VRF
Piano di Qualifica (PdQ)	A	R	-	-	-	V
Design Document (DD)	A	-	C (per coerenza requisiti)	R	C	V
Manuale Utente (MU)	A	-	-	R	C	V
Verbali interni	R/A	R	-	-	-	V
Verbali esterni	R/A	R	-	-	-	V
Test Report (TR)	A	-	-	C	R	V
Documentazione tecnica interna	A	R (per strumenti e template)	R	C	-	V

4.1.6) Definition of Done

La **Definition of Done (DoD)** è un elemento molto importante nello sviluppo software, perché definisce le azioni che devono essere completate affinché i requisiti — espressi tramite un **Product Backlog Item (PBI)** — siano considerati conclusi.

I criteri che la compongono devono essere concreti, verificabili e di dimensione ridotta, e hanno l'obiettivo di garantire un livello minimo di qualità per ogni rilascio o incremento del prodotto.

Di seguito viene riportata la Definition of Done per la fase RTB:

- ☐ Controllare a livello semantico e grammaticale che tutto sia corretto (grammatica, punteggiatura, sintassi, rivedere frasi ripetute/ mal espresse)
- ☐ Controllare di aver incluso tutte le sezioni definite del WoW nel documento su cui si lavora
- ☐ **Nei verbali:** Controllare di aver aggiornato nello status TAB:
 - stato
 - versione
 - ruoli
- ☐ Controllare di aver aggiunto le ultime modifiche anche sulla “tabella delle modifiche del documento”
- ☐ **Nei verbali:** controllare di aver aggiornato la versione nel nome del file
- ☐ **Nei verbali,** controllare che tutte le decisioni corrispondano a issue specifiche nell'issue template.
- ☐ Un documento (o una sua sezione) è considerato completato quando:
 - È stato scritto;
 - È stato verificato;
 - È stata aggiunta una riga nelle tabelle documentarie con il validatore finale.

- ☐ Quando il documento/prodotto è completato, chiudere la issue con

```
git commit -m "commento. Close #numero_issue"
```

Verificare poi effettivamente la chiusura nel Projects Board.

- ☐ Quando tutti i punti sopra sono completati e tutte le issue sono spostate in “Done”:
- Il branch develop può essere unito a main
 - Controllare l’incremento dello sprint corrispondente (e il website)

La seguente **Definition of Done** non è statica, ma dinamica: evolve in base alle esigenze del team di sviluppo.

[Definition-of-done-Guide](#)

4.1.7) Issue tracking System – Guida Operativa

L’**Issue Tracking System** è lo strumento utilizzato dal nostro team di sviluppo per tracciare in maniera efficiente tutte le issue da svolgere e il loro stato di completamento. Il sistema è accessibile a tutti i membri del gruppo attraverso la repository GitHub, dove è disponibile un **template di issue condiviso e centrale**, in modo da evitare incongruenze o confusione.

4.1.7.1) Creazione di una nuova issue

A seguito di verbali interni o esterni, il gruppo decide le attività su cui concentrarsi. L’**amministratore** ha il compito di creare le issue nel sistema utilizzando il **template condiviso**.

Ogni nuova issue deve includere:

1. **Assegnatario/i**

Generalmente è preferibile assegnare la issue a una sola persona. Tuttavia, per attività di formazione o esercitazioni («palestra») si possono assegnare più persone o l’intero gruppo.

2. **Descrizione**

Una spiegazione dettagliata e specifica delle azioni da svolgere.

3. **Scopo**

Indica cosa ci si aspetta di ottenere al termine dell’issue e dove andrà documentato il risultato (ad esempio, in quale documento o sezione del repository).

4. **Autore**

La persona che deve svolgere la issue.

5. **Verificatore**

La persona incaricata di verificare che la issue sia stata risolta correttamente,

in base alla Definition of Done. Il verificatore, a meno di eccezioni, è diversa dall'autore.

6. **Label (ambito/destinazione)**

Questa classificazione consente di organizzare le issue in base al loro ambito o posizione all'interno del progetto:

- Analisi dei requisiti
- Piano di progetto
- Piano di qualifica
- Norme progetto
- Verbale
- Diario di bordo
- Glossario
- Generale -> attività che non rientrano nei documenti sopra: studio di materiale aziendale, lavori sul sito web, gestione repository, attività varie fuori dai documenti principali

7. **Tipo di issue (Type)**

Permette di distinguere la natura delle attività:

- Palestra -> ore formative non rendicontate, attività di ricerca o di studio
- Produttivo -> ore rendicontate con risultati concreti, come la scrittura di documenti da presentare
- Bug -> errori o malfunzionamenti nel codice;
- Correzione -> interventi su documenti o materiali già prodotti per migliorarli, aggiornarli o correggere inesattezze;

[Tracking-your-work-with-issue-Guide](#)

8. **Priorità : Bassa, Media, Alta**

Questa suddivisione ha due scopi:

- ragionare sull'importanza della issue che si sta scrivendo
- comunicare all'assegnatario con quale tempestività dovrà svolgere la issue

9. **Dimensione : ExtraSmall, Small, Medium, Large**

Serve per stimare la mole di lavoro necessaria per portare a termine quella issue.

10. **Data di scadenza**

Normalmente coincide con la fine dello sprint di riferimento.

4.1.7.2) **Flusso operativo**

1. L'amministratore crea una nuova issue tramite il template condiviso.
2. Si assegnano autore/i e verificatore/i.
3. Si compilano descrizione, scopo, label, type, priorità, dimensione, scadenza.
4. La issue viene inserita nello stato iniziale Backlog e segue il flusso fino a Done.

4.1.8) Versionamento

In questa sezione viene spiegata la logica di versionamento dei documenti.

4.1.8.1) Codice di versione

Ogni modifica apportata a un documento genera automaticamente una nuova versione, identificata tramite un codice nel formato:

X.Y.Z

dove ciascuna componente rappresenta uno stato diverso del processo di validazione:

- **X – Versione stabile approvata**

- Indica l'ultima versione ufficialmente approvata dal Responsabile. Il suo incremento segnala una revisione sostanziale o una modifica di grande rilievo. L'incremento di X comporta l'azzeramento automatico di Y e Z.

- **Y – Feature proposta / Approvazione generale**

Rappresenta l'ultima approvazione da parte di un Verificatore. Il suo incremento indica l'introduzione o modifica di una nuova funzionalità o sezione del documento. L'incremento di Y comporta l'azzeramento di Z.

- **Z – Patch della feature proposta / Modifica verificata**

Indica l'ultima modifica di dettaglio verificata (correzioni minori, refusi, aggiustamenti formali). L'incremento di Z rappresenta cambiamenti minori.

4.1.8.1.1) Regole di incremento

- Ogni approvazione genera un incremento della cifra di versione stabile.
- Nel versionamento X.Y.Z, maggiore è il cambiamento, più significativa è la cifra che viene incrementata: X ha un peso maggiore di Y, e Y maggiore di Z
- L'incremento di una cifra comporta sempre l'azzeramento delle cifre alla sua destra, mantenendo coerenza nella progressione delle versioni.

4.2) Gestione dell'Infrastruttura

Questo processo stabilisce e mantiene le infrastrutture che supportano gli altri processi.

4.2.1) Attività previste

1. Implementazione del processo;
2. Creazione;
3. Manutenzione.

Tutte i task legati alle attività di creazione e manutenzione sono di competenza dell'amministratore.

4.2.2) Implementazione del processo

Data la necessità di facilitare il lavoro del gruppo, in particolare la comunicazione asincrona, sono stati adottati diversi strumenti:

Discord: Piattaforma per videoconferenze e messaggistica usata dal gruppo per le riunioni interne da remoto.

Zoom: Piattaforma per videoconferenze usata per le riunioni da remoto con **BlueWind**.

WhatsApp: App di messaggistica istantanea usata per le comunicazioni interne asincrone, tramite un gruppo dedicato alle attività di progetto.

Telegram: App di messaggistica istantanea usata per le comunicazioni asincrone con la proponente, **BlueWind**.

Typst: Linguaggio di markup usato per la creazione dei documenti.

La compilazione è stata automatizzata tramite script.

Tinymist Typst: Estensione di Visual Studio Code per il supporto alla sintassi di Typst e per le funzionalità di live-preview:

Script bash: Sono state implementate diverse automazioni eseguite da GitHub action, il gruppo ha preferito, ove possibile, usare script bash in quanto richiedono solo un terminale linux per eseguire.

Git: Distributed version control system impiegato dal gruppo nello sviluppo di codice e documenti.

GitHub: È una piattaforma basata su cloud che consente agli sviluppatori di archiviare, gestire e collaborare sul codice sorgente dei loro progetti.

È strettamente legato a Git.

Offre molte funzionalità utili alle attività di progetto.

Google docs: Editor online per documenti condivisi, usato dal gruppo per le attività di brainstorming asincrono.

Google sheets: Editor online per fogli di calcolo condivisi, usato dal gruppo per tracciamento di dati.

Google drive: Sistema di file sharing usato per condividere risorse utili oppure ad uso esclusivo del team.

4.2.3) Creazione

In questa sezione viene documentato come sono stati creati e configurati gli strumenti significativi.

Typst

L'ambiente di lavoro typst è stato personalizzato.

Typst offre la possibilità di definire dei template che vengono usati in maniera analoga alle chiamate di funzione di un normale linguaggio di programmazione.

I template più usati svolgono le seguenti funzioni:

Impaginazione:

Permette di impostare facilmente il contenuto di header e footer.

Contiene inoltre le regole di stile comuni a tutti i documenti²

Tabelle:

Sono stati predisposti dei template per la creazione semplificata di tabelle.

Sprint:

Il riassunto degli sprint è un'attività ripetitiva, tramite template vengono effettuati tutti i calcoli necessari e viene predisposto il layout in cui inserire il contenuto.

Marcatatura automatica dei termine del Glossario:

Però va abilitata modificando un apposito flag booleano; se lasciata sempre attiva impatta negativamente le funzionalità di live preview.

Separazione tra contenuto e layout del glossario:

I termini del glossario e le relative definizioni sono definite su un file a parte come dizionario.

Questo permette di automatizzare l'ordinamento e di generare visualizzazioni facilmente personalizzabili (sia in formato pdf che in formato html)

Git

L'intero progetto è usa Git per il versionamento.

Sono stati creati 2 branch principali:

- **Main**, per il rilascio;
- **Develop**, per lo svolgimento delle attività di progetto.

È stato predisposto un .gitignore per evitare la pubblicazione di file indesiderati.

GitHub

È stata creata un [GitHub Organization](#) per le attività di progetto e un [repository](#) per la documentazione.

GitHub Actions:

Sono state configurate delle GitHub Actions per la compilazione automatica dei file Typst e aggiornamento automatico del sito web.

²Ad esempio, i link non sono di colore blu e sottolineati di default. Imponendo questa regola stilistica all'interno del template, questa viene ereditata dal contenuto della pagina.

Strumenti di comunicazione**Strumenti Google****GitHub Pages:**

È stata attivata la funzionalità GitHubPages per l'hosting del sito web.

GitHub Issue tracking system:

Il gruppo ha deciso di avvalersi dell'issue tracking system offerto da GitHub, vedi di più alla **Sezione 4.2.4.1**

Nessuno degli strumenti di comunicazione ha richiesto operazioni significative durante la creazione.

È stata creata una mail apposita per le attività relative al progetto.

Drive e Docs non richiedono particolari operazioni se non il caricamento del materiale. Sheets invece richiede operazioni più complesse per implementare gli indicatori desiderati.

Tabella 1: Strumenti utilizzati

4.2.4) Manutenzione

Nell'ambito del progetto è necessario mantenere aggiornata l'infrastruttura. Le operazioni di manutenzione possono essere di vario tipo.

- In base alla frequenza:
 - Ordinaria, ovvero l'aggiornamento periodico legato alla normale attività di progetto;
 - Non ordinaria, ovvero aggiunte, modifiche e cancellazioni legate a cause non ordinarie.

4.2.4.1) Issue tracking System - Guida Operativa

L'**Issue Tracking System** è lo strumento utilizzato dal nostro team di sviluppo per tracciare in maniera efficiente tutte le issue da svolgere e il loro stato di completamento. Il sistema è accessibile a tutti i membri del gruppo attraverso la repository GitHub, dove è disponibile un **template di issue condiviso e centrale**, in modo da evitare incongruenze o confusione.

4.2.4.2) Creazione e struttura di un issue

Task ordinario. A seguito di riunioni interne o esterne, il gruppo decide le attività su cui concentrarsi. L'**amministratore** ha il compito di creare le issue nel sistema utilizzando un apposito **template**.

Ogni nuova issue deve includere:

1. Assegnatario/i

Generalmente è preferibile assegnare la issue a una sola persona. Tuttavia, per attività di formazione o esercitazioni («palestra») si possono assegnare più persone o l'intero gruppo.

2. Descrizione

Una spiegazione dettagliata e specifica delle azioni da svolgere.

3. Scopo

Indica cosa ci si aspetta di ottenere al termine dell'issue e dove andrà documentato il risultato (ad esempio, in quale documento o sezione del repository).

4. Autore

La persona che deve svolgere la issue.

5. Verificatore

La persona incaricata di verificare che la issue sia stata risolta correttamente, in base alla Definition of Done. Il verificatore, a meno di eccezioni straordinarie, è diversa dall'autore.

6. Label (ambito/destinazione)

Questa classificazione consente di organizzare le issue in base al loro ambito o posizione all'interno del progetto:

- Analisi dei requisiti
- Piano di progetto
- Piano di qualifica
- Norme progetto
- Verbale
- Diario di bordo
- Glossario
- Generale -> attività che non rientrano nei documenti sopra: studio di materiale aziendale, lavori sul sito web, gestione repository, attività varie fuori dai documenti principali.

Nota

Le label potrebbero cambiare durante le attività di progetto, label superflue verranno eliminate, verranno aggiunte nuove label o potrebbero cambiare nome. L'utilità principale delle label è di semplificare la ricerca di issue legati a uno specifico task.

7. Tipo di issue (Type)

Permette di distinguere la natura delle attività e dei task:

- Palestra -> ore formative non rendicontate, attività di ricerca o di studio
- Produttivo -> ore rendicontate con risultati concreti, come la scrittura di documenti da presentare
- Bug -> errori o malfunzionamenti nel codice;
- Correzione -> interventi su documenti o materiali già prodotti per migliorarli, aggiornarli o correggere inesattezze;

[Tracking-your-work-with-issue-Guide](#)

8. **Priorità : Bassa, Media, Alta**

Questa suddivisione ha due scopi:

- ragionare sull'importanza della issue che si sta scrivendo
- comunicare all'assegnatario con quale tempestività dovrà svolgere la issue

9. **Dimensione : ExtraSmall, Small, Medium, Large**

Serve per stimare la mole di lavoro necessaria per portare a termine quella issue.

10. **Data di scadenza**

Normalmente coincide con la fine dello sprint di riferimento.

4.2.4.2.1) Flusso operativo

1. L'amministratore crea una nuova issue tramite il template.
2. Si assegnano autore/i e verificatore/i.
3. Si compilano descrizione, scopo, label, type, priorità, dimensione, scadenza.
4. La issue viene inserita nello stato iniziale Backlog e segue il flusso fino a Done.

4.2.4.2.2) Project board

Le issue vengono inserite in un'apposita project board per riflettere meglio il quadro generale.

4.2.4.2.3) Milestone

Le milestone sono una parte fondamentale nella pianificazione di lungo periodo.

È compito dell'amministratore assegnare gli issue alla milestone. Deve anche tenere le milestone aggiornate in caso di cambiamenti.

4.2.4.3) GitHub Actions

Le GitHub Actions sono configurabili e modificabili tramite file YML.

Sono salvate nella cartella `.github/workflows`.

4.2.4.4) Script e automazioni

Il mantenimento degli script è compito dell'amministratore.

Questo avviene quando viene deciso di introdurre una nuova automazione, un'automazione diventa obsoleta o un'automazione è errata.

Le automazioni strettamente legate alla redazione di documenti Typst sono salvate in un'apposita cartella. Le automazioni strettamente legate alla redazione di documenti Typst, i template, sono salvate in un'apposita cartella `src/TypstTemplate` all'interno del repository [Documentazione](#).

Script a scopo generale sono salvati nella cartella `scripts`.

4.2.4.5) Discord

Le uniche operazioni degne di nota per la piattaforma Discord sono la creazione di nuovi canali e la moderazione di bassa entità.

4.2.4.6) Strumenti Google

Di seguito sono esposte le attività e i task relative agli strumenti Google.

Google drive:

Eliminazione di file obsoleti e creazione di file.

Google docs:

Dove opportuno predisporre l'opportuno layout per il documento.

Google Sheets:

Implementare le metriche stabilite tramite le funzionalità dei fogli di calcolo.

4.3) Processo di miglioramento

Il processo di miglioramento si occupa di stabilire, valutare, misurare, controllo e migliorare i processi di ciclo di vita.

4.3.1) inizializzazione del processo

Questa attività si occupa di definire e documentare i processi organizzativi.

Questi processi permettono di semplificare le attività di progetto.

Questo documento svolge tale compito.

4.3.2) Valutazione del processo

Questa attività consiste nello sviluppo, documentazione, uso e storicizzazione³ di procedure di valutazione.

Per garantirne l'efficacia e l'efficienza è necessaria una revisione periodica.

Per maggiori dettagli si veda la **Sezione 6**.

4.3.3) Miglioramento del processo

Come conseguenza delle attività di valutazione è necessario porre in essere e documentare l'attività di miglioramento.

È utile raccogliere e analizzare i vari dati relativi al processo:

- Storia;
- Dati tecnici;
- Valutazioni;
- Performance di processo.

Al fine di dare un miglior resoconto del processo e degli aspetti da migliorare.

4.3.4) Ciclo PDCA

Queste attività costituiscono il ciclo

- **Plan:** Definizione delle procedure e degli standard (**Sezione 4.3.1**);
- **Do:** Applicazione operativa dei processi nei progetti attivi;
- **Check:** Misurazione delle performance e Audit (**Sezione 4.3.2**);

³Tramite registrazione

- **Act:** Aggiornamento delle Norme di Progetto e ottimizzazione delle procedure (Sezione 4.3.3).



Figura 1: Rappresentazione del ciclo PDCA applicato ai processi organizzativi

4.4) Processo di formazione

Questo processo si occupa di mantenere il personale ben allenato e in grado di svolgere i compiti assegnati.

4.4.1) Avvio del processo

Per poter avviare e aggiornare il processo di formazione vanno considerate varie informazioni:

- Tipo e livello di formazione necessari;
- Categorie del personale da formare;
- Le risorse disponibili, i bisogni e la pianificazione in termini di tempo.

4.4.2) Sviluppo

È necessario fornire materiale per la formazione.

4.4.3) Esecuzione

Vanno tenuti dei registri sulla formazione.

4.4.4) Apprendimento libero

Non sempre un programma di apprendimento così rigoroso riesce a rispondere in modo efficace alle esigenze reali e immediate che un membro del team può incontrare nello svolgimento di un task.

Perciò è lasciata anche libertà di formazione autonoma.

5) Metriche e standard per la Qualità

6) Metriche di Qualità del Processo

7) Metriche di Qualità del Prodotto

8) Best Practices

In questa sezione vengono riportate le best practices e pratiche standard concordate col gruppo al fine di garantire coerenza all'interno dei documenti.

8.1) Formato nome dei verbali

Al fine di avere ordine estetico all'interno della repo, è stato deciso di adottare il seguente standard per la nomina dei verbali. Di questi documenti interessa data e versione, dunque saranno nel formato:

YYYY-MM-DD_Verbale-vX.Y.Z.typ

8.2) Scrittura dei commit

[The-art-of-writing-meaningful-git-commit-messages-fonte](#)

I commit dovrebbero avere un tipo ed una descrizione: il tipo indica qual è l'obiettivo del commit, mentre la descrizione aiuta il lettore a comprendere meglio quali cambiamenti sono stati effettuati.

Le regole generali sono:

- iniziare il commit con tipo seguito da «:» .
- lasciare uno spazio tra tipo e descrizione.
- iniziare la descrizione con una lettera maiuscola.
- limitare la descrizione a massimo 50 caratteri.
- indicare alla fine del commit la issue a cui ci si sta riferendo con:

```
git commit -m "Commento.Issue #01"
```

Nel caso sia necessario modificare un commit (ad esempio in caso di errori) si utilizza il seguente comando

```
git commit --amend
```

N.B. : È consigliato l'utilizzo del comando per modificare commit in locale prima di fare push nella repository condivisa. È preferibile astenersi dal modificare commit che sono già stati resi pubblici.

8.3) Issue Template